

20-S2-Q3

Solution (a) Partially-Mapped Cross

PMX is a permutation-preserving operator that guarantees offspring are legal tours.

Step	Example with 10-city tour
Parents	$P_1 = \langle 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9\ 10 \rangle$ $P_2 = \langle 4\ 1\ 2\ 8\ 7\ 6\ 10\ 3\ 5\ 9 \rangle$
1 Choose two cut points	positions 3 and 7 (inclusive).
2 Copy the segment of P_1 into Child 1	Segment $\langle 3\ 4\ 5\ 6\ 7 \rangle$ placed at the same positions: $C_1 = \mathbf{3\ 4\ 5\ 6\ 7}$
3 Build the mapping table	$(3 \leftrightarrow 2, 4 \leftrightarrow 8, 5 \leftrightarrow 7, 6 \leftrightarrow 6, 7 \leftrightarrow 10)$ — derived by pairing genes in the cut segment of P_1 with those at the same positions in P_2 .
4 Fill the remaining slots of Child 1	Scan P_2 from the left; every city not yet in C_1 is considered. If the city is already present (directly or via the mapping), keep tracing through the mapping until a vacant city is found. • First position: P_2 has 4 (already in child) $\rightarrow$ map $4 \rightarrow 8 \rightarrow 8$ is free $\rightarrow C_1[1]=8$. • Second position: P_2 has 1 (free) $\rightarrow C_1[2]=1$. • ... continue until all blanks are filled.
Resulting offspring	$C_1 = \langle 8\ 1\ 3\ 4\ 5\ 6\ 7\ 2\ 10\ 9 \rangle$. (The complementary child is produced symmetrically.)

Because every city appears exactly once, PMX maintains feasible tours while passing contiguous subsequences from parents and respecting positional information – properties that make it popular for GA-based TSP solving

(b) Neighborhood-based Differential Evolution (NbDE) for Niching

Goal: identify many peaks of a multi-modal function in one run.

Canonical DE component	NbDE modification for niching
Mutation $v_i = x_{r1} + F(x_{r2} - x_{r3})$ where $r1, r2, r3$ are distinct random indices.	<i>Neighborhood selection</i> – choose $r1, r2, r3$ only from individuals within a radius RR around the target x_{i-1} (measured in decision space). This yields several semi-independent “sub-populations” around different peaks.
Crossover	No change, but a <i>crowding replacement</i> rule is applied: the trial vector competes only with its closest neighbor; if fitter, it replaces that neighbor.

Canonical DE component	NbDE modification for niching
Adaptive radius RR	Start large to detect major modes; gradually shrink RR so each niche tightens around its optimum.
Elitist archiving	Maintain an archive of best solutions per niche; a new candidate enters the archive if it is inside radius RaR_a of an archive member and fitter than it, otherwise it may create a new niche.
Re-initialisation	When the local variance inside a niche falls below ϵ , a few worst individuals are re-initialized to unexplored regions to search for missing peaks.

These modifications keep search operations local (so multiple niches evolve in parallel) while the crowding replacement and adaptive radii preserve diversity and prevent niche take-over by dominant optima.

(c) Multisurface Proximal Support Vector Machine (MPSVM)

MPSVM constructs **two** non-parallel proximal hyperplanes, each one closer to the points of one class and as far as possible from the other class.

Training formulation

Given labelled data $\{(x_i, y_i)\}_{i=1}^m$, $y_i \in \{+1, -1\}$

split the matrices as

$$A = \{x_i^\top \mid y_i = +1\} \in \mathbb{R}^{m_+ \times n}, \quad B = \{x_i^\top \mid y_i = -1\} \in \mathbb{R}^{m_- \times n}.$$

For class +1 we seek a hyperplane

$$H_1 : w_1^\top x + b_1 = 0$$

that is **proximal** to the +1 samples and distant from the -1 samples.

MPSVM solves the following pair of *quadratic* optimization problems (written here in matrix form without slack for clarity):

$$\begin{aligned} \min_{(w_1, b_1)} \quad & (Aw_1 + \mathbf{1}b_1)^\top (Aw_1 + \mathbf{1}b_1) \\ \text{s.t.} \quad & (Bw_1 + \mathbf{1}b_1)^\top (Bw_1 + \mathbf{1}b_1) = 1 \end{aligned}$$

$$\begin{aligned} \min_{(w_2, b_2)} \quad & (Bw_2 + \mathbf{1}b_2)^\top (Bw_2 + \mathbf{1}b_2) \\ \text{s.t.} \quad & (Aw_2 + \mathbf{1}b_2)^\top (Aw_2 + \mathbf{1}b_2) = 1 \end{aligned}$$

Each problem reduces to a **generalized eigen-value** (GEV) task:

$$\begin{bmatrix} 0 & A \\ A^\top & 0 \end{bmatrix} \begin{bmatrix} w_1 \\ b_1 \end{bmatrix} = \lambda \begin{bmatrix} B^\top B & B^\top \mathbf{1} \\ \mathbf{1}^\top B & m_- \end{bmatrix} \begin{bmatrix} w_1 \\ b_1 \end{bmatrix},$$

and analogously for (w_2, b_2) .

The eigenvector associated with the **smallest positive eigen-value** gives the desired hyperplane parameters.

Classification rule

For a test point x

$$d_1 = |w_1^\top x + b_1|, \quad d_2 = |w_2^\top x + b_2|.$$

Assign the point to the class of the *closer* hyperplane:

$$\hat{y} = \begin{cases} +1, & d_1 < d_2, \\ -1, & \text{otherwise.} \end{cases}$$

Properties

- **Non-parallel margins** – unlike standard SVM's single maximum-margin hyperplane, MPSVM uses two proximal surfaces, which can be advantageous when classes are not linearly separable by one flat margin.
- **Fast training** – both surfaces come from solving two small GEV problems (size = $n + 1$), avoiding large QP's.
- **Kernelisation** – replacing dot products by kernels in the GEV matrices yields a nonlinear MPSVM suitable for complex boundaries.

Thus MPSVM forms an efficient two-class classifier that often rivals (and sometimes surpasses) traditional SVMs in speed while providing comparably accurate decision boundaries.